战队名称
安全破壁者
比赛排名
总分数 4897
学生组排名 7
总排名 7
战队成员 1 李睿娴
战队成员 2 王桂玉
战队成员 3 王昊鸣

一、easyAES

1、原代码中，随机生成两个字节然后重复 16 次得到 key，随机生成 16 字节的 iv，输出 key 和 iv 异或后的结果。然后采用 AES 加密，CBC 模式，对 flag 进行加密。

2、解题重点在于 key 和 iv 的值，因为 key 只有两字节随机，可以直接暴力破解，然后根据异或值反解出 iv 值。尝试用 key 和 iv 值求解，看看是不是有效文本。（因为头固定，大概率是 CTF{}，flag{}或者 BITsCTF{}之类的）如果符合要求就输出解密的结果。

代码如下：

```
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes, bytes_to_long
import string
def is_printable(data):
    try:
        decoded = data.decode('utf-8', errors='ignore')
        return all(c in string.printable for c in decoded)
    except:
        return False
def brute_force_enhanced(xor_result, enc_flag):
    found = []

    for i in range(65536):
        key_candidate = long_to_bytes(i, 2) * 16

        key_int = bytes_to_long(key_candidate)
        iv_int = key_int ^ xor_result
        iv_candidate = long_to_bytes(iv_int, 16)

        try:
            aes = AES.new(key_candidate, AES.MODE_CBC, iv_candidate)
            decrypted = aes.decrypt(enc_flag)

            if (b'flag{' in decrypted or
                    b'FLAG{' in decrypted or
                    b'ctf{' in decrypted or
                    is_printable(decrypted)):
```

```
                print(f"Key (hex): {key_candidate.hex()}")
                print(f"IV (hex): {iv_candidate.hex()}")
                print(f"Decrypted: {decrypted}")

                found.append((key_candidate, iv_candidate, decrypted))
                return found

        except Exception as e:
            continue

    return found
if __name__ == "__main__":
    xor_value = 54755426038978086460310747316994556563126341157253266647920514387148480709140
    encrypted_flag =
b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

    results = brute_force_enhanced(xor_value, encrypted_flag)

    if not results:
        print("No valid key found!")
```

二、babyRSA

1、RSA 加密。Flag 被分成两部分，转换成整数，然后计算 c1 和 c2.再用 RSA 加密 c1、c2 得到 m1、m2.

2、Secret.txt 里给了 N、m1、m2，分解 N 得到 p、q，解密得到 c1、c2，然后恢复 F1、F2 即可得到 flag。

代码如下：

```
import math
from Crypto.Util.number import isPrime, long_to_bytes, inverse

N =
636585149594574746909030160182690866222909256464847291783000651837227921337237899651287943597
773270944384034858925295744880727101606841413640006527614873110651410155893776548737823152943
797884729130149758279127430044739254000426610922834573094957082589539445610828279428814524313
491262061930512829074466232633130599104490893572093943832740301809630847541592548921200288222
432789208650949937638303429456468889100192613859073752923812454212239908948930178355331390933
536771065791817643978763045030833712326162883810638120029378337092938662174119747687899484603
628344079493556601422498405360731958162719296160584042671057160241284852522913676264596201906
163
m1 =
437018910555377754758713090674828434552623609418189605883878395265943244846522356956125593648
928796885738904275195790322403904731577406221662800615649192135096594758567990700774710458621
606544804995324319230202175408806666706760877979225732807083632781339065831416759856459922014
414321419450650531126833451929640356477652908076978264462932182064696092260867124176080216707
```

2303185824892016429186046788968005318868836587626927114841136652111457994151509557942312034753269100507145529907890663063975000975106883318093805875380417352806523685326674344204327480555626216929911748510640176509890886868972962874987772280199858697989560677638118791788299559943366

m2 =

4237751568873457714832140731745635662259718993781379731133003300507448062916005010304393004930073734395724865768549003771839005681248756485804051179003347635507347048505455251953122822860189186926267667546987157976752426499878301232540750748492601324652727990505357156127909059250768098083685196252399118126351744996553309467488781702825544912208737301055942189975538397728280686886764468787475609711428909966476158484022872818840934068260174118388896092498636773178523365244974569960750221309560470600275037915714286453227133778163443489032240593311116566239168855132482884514907219444505525063000466060508375157092179645411770435125441473274397933716612

```
def factor_n(n):
    sqrt_n = math.isqrt(n)
    p = sqrt_n
    while not isPrime(p):
        p -= 1
    while n % p != 0:
        p -= 1
        while not isPrime(p):
            p -= 1
    q = n // p
    return p, q

p, q = factor_n(N)
print(f"p = {p}")
print(f"q = {q}")

e = 0x10001
phi = (p - 1) * (q - 1)
d = inverse(e, phi)
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)
print(f"c1 = {c1}")
print(f"c2 = {c2}")

s = c1
p_val = (s**3 - c2) // (3 * s)

discriminant = s**2 - 4 * p_val
if discriminant < 0:
    raise ValueError("判别式为负，无实数解")
```

```
sqrt_disc = math.isqrt(discriminant)
if sqrt_disc**2 != discriminant:
    raise ValueError("判别式不是完全平方数")
F1 = (s + sqrt_disc) // 2
F2 = (s - sqrt_disc) // 2
flag1 = long_to_bytes(F1)
flag2 = long_to_bytes(F2)
flag = flag2 + flag1
print(f"Flag: {flag.decode()}")
```

三、Remaining thing

这题用到中国剩余定理和低指数攻击（e=3）。通过不同的模数，利用中国剩余定理求解方程组，可以恢复明文。

代码如下：

```
from sympy.ntheory.modular import crt
from sympy import integer_nthroot

e = 3

N1 =
19831146070473780450608271965513333016477420167810804693209535211751519378798100050738385201161854775668250184491612993710844933532155638787045219486707787420282058177611838960790320734624809796310926894615507014393905439164108305343147056285187048 03
C1 =
13192445407076444144417541514427188094854353253584529650871742429298429176122106888115309616831240701274243771913770298764454625711248881032556304446303580561665126886670936043604468812024111116443358272674121554396071146617853939756537893431091839 55

N2 =
66111404265300574613935728983655950688111510106384565086238514901421964133746188531859108773677252707542147423426687722933483143467420638917923485639731545533291089547341393165686480533491924103483747009620183498000762679606287593577239113476062047 33
C2 =
41998123843875621116729989501531316956049116280732482300490270393719879500976089055100523216197820417935038248527006621265771328125158252787890936500616657191006849984874487887897908316503588758727613717876734500568394557092081398962537850559205431 04

N3 =
14126749535297606509514418674348551918358936748316341471639452521057767873962355396372423101375036984635365376397773384541330304056627522956980062104259147867100357268271782395123516284699902354599667106061620180566549332990378692446782196699116141 17
C3 =
41663026452800366344084116174180504236555705080505184877204086431216138707209002745841467383559886160630317645159881729713488002376046452536102884351087721916511569274797438511760002651 1
```

```
746387746634185154914717287334233494293011261623790319302692124


x, modulus = crt([N1, N2, N3], [C1, C2, C3])


m, is_exact = integer_nthroot(x, 3)


if is_exact:
    hex_str = hex(m)[2:]
    if len(hex_str) % 2:
        hex_str = '0' + hex_str
    flag = bytes.fromhex(hex_str).decode('utf-8')
    print(flag)
else:
    print("无法找到精确的立方根")
```

四、Math

1、 拖入 IDA 分析原代码，输入 abcdef 的值后，经过 judge 函数处理，然后输出。

```
  _main(argc, argv, envp);
  puts(&::Buffer);
  putchar(10);
  printf("a = ");
  if ( scanf("%u", &a) != 1 )
    return 0;
  printf("b = ");
  if ( scanf("%u", &b) != 1 )
    return 0;
  printf("c = ");
  if ( scanf("%u", &c) != 1 )
    return 0;
  printf("d = ");
  if ( scanf("%u", &d) != 1 )
    return 0;
  printf("e = ");
  if ( scanf("%u", &e) != 1 )
    return 0;
  printf("f = ");
  if ( scanf("%u", &f) != 1 )
    return 0;
  if ( (unsigned __int8)judge(a, b, c, d, e, f) )
  {
    puts("Right.");
    F = f;
    E = e;
    LODWORD(int_d) = d;
    LODWORD(int_c) = c;
    sprintf(Buffer, "BITs2CTF{%x%x%x%x%x%x}", a, b, int_c, int_d, E, F);
    puts(Buffer);
  }
  else
  {
    puts("Wrong.");
  }
  return 0;
}
```

2、查看 judge 函数代码，对变量进行一系列加减乘和异或操作。

```
_BOOL8 __fastcall judge(unsigned int a, unsigned int b, unsigned int c, unsigned int d, unsigned int e, unsigned int f
{
  int v7; // [rsp+24h] [rbp-2Ch]
  unsigned int v8; // [rsp+28h] [rbp-28h]
  int v9; // [rsp+2Ch] [rbp-24h]
  int v10; // [rsp+30h] [rbp-20h]
  int v11; // [rsp+34h] [rbp-1Ch]
  int v12; // [rsp+38h] [rbp-18h]
  unsigned int v13; // [rsp+3Ch] [rbp-14h]
  int v14; // [rsp+40h] [rbp-10h]
  unsigned int v15; // [rsp+44h] [rbp-Ch]
  unsigned int v16; // [rsp+48h] [rbp-8h]
  unsigned int v17; // [rsp+4Ch] [rbp-4h]

  if ( a <= 0x186A0 || b <= 0x186A0 || c <= 0x186A0 || d <= 0x186A0 || e <= 0x186A0 || f <= 0x186A0 )
    return 0i64;
  if ( a > 0xF423F || b > 0xF423F || c > 0xF423F || d > 0xF423F || e > 0xF423F || f > 0xF423F )
    return 0i64;
  v17 = (unsigned int)sum(a, b) % 0xE8329;
  v16 = jian(a, b);
  v15 = cheng(2u, c);
  v14 = sum(v16, v15);
  v13 = cheng(4u, f);
  v12 = XOR(v13, d);
  v11 = jian(d, e);
  v10 = cheng(5u, v11);
  v9 = sum(a, f);
  v8 = cheng(3u, d);
  v7 = jian(v8, v11);
  return v17 == 597141 && v14 == 1644082 && v12 == 1161537 && v10 == 343890 && v9 == 1136538 && v7 == 1952901;
}
```

3、把函数丢给 ai 解，解得 abcdef 的值，调试程序，在出口设断点，输入 abcdef 的值后得到 flag。

```
_BOOL8 __fastcall judge(unsigned int a1, unsigned int a2, unsigned int a3, unsigned
int a4, unsigned int a5, unsigned int a6)
{
  int v7; // [rsp+24h] [rbp-2Ch]
  unsigned int v8; // [rsp+28h] [rbp-28h]
  int v9; // [rsp+2Ch] [rbp-24h]
  int v10; // [rsp+30h] [rbp-20h]
  int v11; // [rsp+34h] [rbp-1Ch]
  int v12; // [rsp+38h] [rbp-18h]
  unsigned int v13; // [rsp+3Ch] [rbp-14h]
  int v14; // [rsp+40h] [rbp-10h]
  unsigned int v15; // [rsp+44h] [rbp-Ch]
  unsigned int v16; // [rsp+48h] [rbp-8h]
  unsigned int v17; // [rsp+4Ch] [rbp-4h]

  if ( a1 <= 0x186A0 || a2 <= 0x186A0 || a3 <= 0x186A0 || a4 <= 0x186A0 || a5 <=
0x186A0 || a6 <= 0x186A0 )
    return 0i64;
  if ( a1 > 0xF423F || a2 > 0xF423F || a3 > 0xF423F || a4 > 0xF423F || a5 >
0xF423F || a6 > 0xF423F )
    return 0i64;
  v17 = (unsigned int)fun1(a1, a2) % 0xE8329;
  v16 = fun2(a1, a2);
  v15 = fun3(2u, a3);
  v14 = fun1(v16, v15);
  v13 = fun3(4u, a6);
  v12 = fun4(v13, a4);
  v11 = fun2(a4, a5);
  v10 = fun3(5u, v11);
  v9 = fun1(a1, a6);
  v8 = fun3(3u, a4);
  v7 = fun2(v8, v11);
  return v17 == 597141 && v14 == 1644082 && v12 == 1161537 && v10 == 343890 &&
v9 == 1136538 && v7 == 1952901;
}unsigned __int64 __fastcall fun1(unsigned int a1, unsigned int a2)
{
  int v3; // [rsp+8h] [rbp-18h]
  int v4; // [rsp+Ch] [rbp-14h]
  char v5; // [rsp+10h] [rbp-10h]
  unsigned int v6; // [rsp+14h] [rbp-Ch]
  __int64 v7; // [rsp+18h] [rbp-8h]

  v7 = 0i64;
  v6 = 0;
  v5 = 0;
  while ( a1 || a2 )
  {
    v4 = a1 & 1;
    v3 = a2 & 1;
```

```
    a1 >>= 1;
    a2 >>= 1;
    v7 |= (unsigned __int64)(((unsigned __int8)v3 ^ (unsigned __int8)(v4 ^ v6)) & 1)
<< v5;
    v6 = v4 & (v3 | v6) | v3 & v6;
    ++v5;
  }
  return v7 | ((unsigned __int64)v6 << v5);
}__int64 __fastcall fun3(unsigned int a1, int a2)
{
  char v3; // [rsp+8h] [rbp-8h]
  unsigned int v4; // [rsp+Ch] [rbp-4h]

  v4 = 0;
  v3 = 0;
  while ( a1 )
  {
    if ( (a1 & 1) != 0 )
      v4 += a2 << v3;
    a1 >>= 1;
    ++v3;
  }
  return v4;
}__int64 __fastcall fun4(unsigned int a1, unsigned int a2)
{
  char v2; // al
  int v4; // [rsp+0h] [rbp-10h]
  int v5; // [rsp+4h] [rbp-Ch]
  int v6; // [rsp+8h] [rbp-8h]
  unsigned int v7; // [rsp+Ch] [rbp-4h]

  v7 = 0;
  v6 = 0;
  while ( a1 || a2 )
  {
    v5 = a1 & 1;
    v4 = a2 & 1;
    a1 >>= 1;
    a2 >>= 1;
    v2 = v6++;
    v7 += (v5 ^ v4) << v2;
  }
  return v7;
}__int64 __fastcall fun2(unsigned int a1, int a2)
{
  return fun1(a1, -a2);
}
```

已思考（用时 207 秒） ›

根据代码分析，需要通过 `judge` 函数验证六个输入值（a, b, c, d, e, f），这些值必须在100000（0x186A0）到1000000（0xF423F）之间。通过解方程得到以下值：

- a = 678737 (十六进制: a5b51)
- b = 869485 (十六进制: d446d)
- c = 917415 (十六进制: dffa7)
- d = 673893 (十六进制: a4865)
- e = 605115 (十六进制: 93bbb)
- f = 457801 (十六进制: 6fc49)

```
请输入6个六位数: a, b, c, d, e, and f.

a = 678737
b = 869485
c = 917415
d = 673893
e = 605115
f = 457801
Right.
BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}
```

https://gemini.google.com/share/9e12baeb6071
https://gemini.google.com/share/a06b179ef876
https://gemini.google.com/share/b693efe99c91